Probni kolokvij

Web programiranje

Napomena: Na kolokviju je dopušteno koristiti materijale s nastave te sve dostupne izvore s neta. Nije dopušteno međusobno komunicirati. Rješenja se provjeravaju programom za otkrivanje plagijata. ~~Komprimirano rješenje `prezime_ime_kolokvij_1.zip` pošaljite nastavniku na Teams.~~ Rješenje prvog zadatka treba izgledati kao na `zad_1.png`. Rješenje drugog zadatka treba izgledati i ponašati se kao na `zad_2_1.png`, `zad_2_2.png` i `zad_2.mp4`.

~~Zadatak 1 (30 bodova)~~

Za predložak koristite projekt `custom_bootstrap`. Nadogradite predložak tako da se uz postojeće JavaScript biblioteke u klijent automatski dodaje i jQuery *(3 boda)*. Prilagodite Bootstrap tako da je primarna boja `#3e5eaf` *(1 bod)*. Postavite font klijenta na Kanit *(2 boda)*. Korištenjem npm-a, dodajte Bootstrap Icons i prema naputcima u dokumentaciji, dodajte u projekt *(1 bod)*.

Stvorite poslužiteljski dio aplikacije u kojem ćete inicijalizirati Express.js poslužitelj koji sluša na portu `3000` *(2 boda)* ~~te kojem je statična mapa ona u kojoj se nalazi klijent *(1 bod)*~~. Dodajte vanjski modul `todos.js` u kojem će se nalaziti polje TODO-ova (kopirati iz `copy.txt`) koje je potrebno unijeti u glavni dio poslužiteljske aplikacije *(2 boda)*. Definirajte GET rutu koja prema klijentu šalje sve TODO zapise *(2 boda)*.

Na klijentu u HTML-u stvorite navigacijsku traku kao na snimkama koja sadrži table ikonu uz natpis „Todo App“ s odgovarajućim stilom dobivenim isključivo preko Bootstrap klasa *(5 bodova)*.

Na klijentu u HTML-u stvorite tablicu s odgovarajućim stupcima i odgovarajućim Bootstrap klasama[1] *(3 boda)* čiji su retci u `tbody` dinamički popunjeni sa zapisima dohvaćenim sa poslužitelja. U tu svrhu u `script.js` definirajte i iskoristite funkcije `getTodos` *(3 boda)* te `renderTableRows` *(5 bodova)* pri čemu ćete pri pozivu prve funkcije njoj proslijediti drugu kao *callback*. To znači da druga funkcija treba biti pozvana iz prve te joj se trebaju proslijediti dohvaćeni TODO zapisi. Za formatiranje datuma i vremena TODO zapisa koristite funkciju

```
const prettyDate = (dateStr) => {
    const formatter = new Intl.DateTimeFormat("en-US", {
        dateStyle: "short",
        timeStyle: "short"
    });
    return formatter.format(new Date(dateStr));
}
```

Ukoliko je atribut `completed` jednak `true`, ikona treba biti check-circle-fill, a u suprotnom x-circle-fill.

[1] Sadržaj stupca “Completed” mora biti centriran.

~~Zadatak 2 (30 bodova)~~

Nadogradite izgled tablice tako da je nakon stupca „Completed“ prikazan stupac “Modify” čiji je sadržaj također centriran te u čijoj se svakoj ćeliji nalazi ikona pencil-square *(1 bod)*. Na poslužitelju uz svaki todo zapis dodajte jedinstveni identifikator `id` korištenjem biblioteke `uuid` ili `crypto` *(2 boda)*.

Proučite dokumentaciju za Bootstrap Modal. U HTML-u nakon svog prikazanog sadržaja smjestite predložak za modal koji ne treba imati naslov, u sadržaju treba biti smještena forma koja sadrži dvije floating label kontrole forme za opis (`type="text"`) i vrijeme (`type="datetime-local"`) te checkbox za to je li TODO završen ili ne pri čemu sve kontrole forme trebaju imati odgovajuće atribute u skladu s atributima svakog TODO zapisa *(6 bodova)*. U podnožju modala trebaju biti gumbovi kao u primjerima iz dokumentacije. Za potrebe testiranja možete se poslužiti gumbom za otvaranje modala kao u dokumentaciji.

Instancirajte zatim instancu klase `Modal` koja predstavlja modal kao globalnu varijablu u `script.js` *(1 bod)*. Uvezite tu klasu s `import { Modal } from "bootstrap";`. Definirajte funkciju `hide` koja će zatvoriti modal *(1 bod)* i koja je *click event handler* za gumb “Close”[2] *(1 bod)*.

Definirajte funkciju `openEditModal` *(5 bodova)* koja prihvaća `todo`, informacije zatim dodjeljuje kontrolama forme (za prva dva inputa možete koristiti jQuery val, a za checkbox jQuery prop)[3]. Također, bitno je sačuvati vrijednost `id`-ja zapisa koja je nadalje potrebna za uređivanje zapisa što možete riješiti ili globalnom varijablom ili da tu vrijednost pridružite skrivenoj kontroli forme. Nakon popunjavanja vrijednosti forme potrebno je otvoriti modal.

Svakoj ćeliji u kojoj se nalazi ikona za uređivanje pridružite event handler koji će pozivati `openEditModal` s prosljeđenim TODO zapisom (najjednostavnije tako da definirate tu klikabilnu ćeliju s jQuery-jem te definirate event s jQuery on) *(3 boda)*.

Na poslužitelju definirajte PUT rutu za nadograđivanje koja prihvaća TODO zapis[4] te ga mijenja u polju *(3 boda)*. Na klijentu definirajte funkciju `submit` *(5 bodova)* koja će parsirati vrijednosti forme, složiti nadograđeni TODO zapis, poslati ga prema poslužitelju (možete koristiti jQuery ajax)[5], a nakon uspješnog odgovora s poslužitelja, potrebno je ponovno učitati sve zapise te zatvoriti modal. `submit` postavite kao *click event handler*[6] za gumb “Save changes” *(1 bod)*.

Prije komprimiranja konačnog rješenja obrišite mapu `node_modules` *(1 bod)*.

---

[2] Primijetite da tu funkciju morate pridružiti globalnom objektu `window` (npr. `window.hide = hide;`) kako bi ona bila vidljiva u UI dijelu aplikacije, odnosno kako bismo ju mogli pozvati u HTML-u.

[3] Osvrnite se na primjere za postavljanje, a ne za dohvaćanje vrijednosti.

[4] Stoga je potrebno uključiti middleware-e za parsiranje podataka s klijenta `app.use(express.json())` i `app.use(express.urlencoded({ extended: false }))`.

[5] U pozivu postavite `contentType: 'application/json'` te podatke koje šaljete mapirajte s `JSON.stringify` kako ne bi došlo do neželjene pretvorbe Booleovih varijabli u stringove.

[6] `submit` morate pridružiti objektu `window`.